

AIQT Milestone 16 Build Specification

M16 - Project Initialization UX and Root Configuration - v0.2

Document status: Revised draft for review before implementation.

Milestone type: Product hardening / startup UX / root configuration.

Created: 2026-07-15

1. Milestone Summary

M16 makes AIQT startup usable with a short project instruction. A user should be able to tell an agent: "Project idea: ... Use the AIQT CLI for this project. Create a new folder and run aiqt init to start." AIQT should then infer or capture the implementation root and inject root discipline into its own guidance.

- Primary finding addressed: F064 - AIQT startup flow requires too much manual root and operating-context instruction.
- Core outcome: implementation root becomes an explicit operational concept backed by the existing project field `existingRepositoryPath`.
- Default UX: one folder acts as AIQT control root, implementation root, and Git repository root.
- Advanced UX: split-root projects are supported by setting implementation root through a clearer CLI alias.

2. Source Alignment

This milestone extends the AIQT MVP Specification v0.6 and Technical Architecture Specification v0.3. It preserves the governing pipeline:

```
state -> graph -> packet -> checkpoint -> review/export -> guided workflow -> agent driver ->
management/recovery -> design guidance -> operating discipline -> source-control discipline ->
implementation repository boundary discipline -> initialization/root configuration
```

M16 does not change AIQT from a workflow controller into an implementation executor. It improves how AIQT stores and renders operating context.

3. Dogfood Finding

ID	Finding	Observed Impact	Desired Fix
F064	AIQT startup flow requires too much manual root and operating-context instruction.	Agents need long manual prompts for control root, implementation root, Git boundary, app command location, and Playwright location. This causes wrong-folder app creation, parent Git usage, and repeated setup friction.	AIQT init/update should capture or resolve the implementation root and generated prompts/packets should inject it automatically.

4. Product Goal

After M16, the expected first agent prompt for a normal project should be small:

```
Project idea:
<project idea>

Use the AIQT CLI for this project.
Create a new folder and run aiqt init to start.
Use AIQT to plan, implement, checkpoint, review, and export the project.
```

The AIQT CLI should supply detailed root, source-control, working-directory, checkpoint, and packet discipline through generated guidance, without requiring the user to repeat paths in every prompt.

5. Key Design Decision: Reuse Existing Repository Path

M16 must not create a parallel root model. The current project schema already has `existingRepositoryPath` and existing CLI support through `aiqt update --repository-path`. M16 redefines that field operationally rather than adding `paths.implementationRoot`.

- `existingRepositoryPath` is the canonical persisted implementation root field.
- Implementation root means: the directory where application source code, Git, validation commands, and Playwright/browser checks run.
- If `existingRepositoryPath` is unset, resolved implementation root falls back to the runtime control root.
- `controlRoot` is runtime-derived from the loaded `.aiqt` location and must not be persisted as an absolute path in `project.json`.
- `--implementation-root` is a clearer alias for writing `existingRepositoryPath`, not a new storage path.

6. Behavioral Position in Workflow

- `aiqt init` is the startup entry point for new projects.
- If no implementation root is provided, `implementationRoot` resolves to the current AIQT control root.
- The AIQT control root is the directory containing `.aiqt` and is derived at runtime.
- Generated guidance must name both the control root and resolved implementation root when available.
- AIQT must not require users to repeat root paths in every agent prompt once they are configured.
- M16 strengthens prompt/packet quality; it does not run Git, create GitHub repositories, or execute app commands.

7. Scope

7.1 In Scope

- Clarify `existingRepositoryPath` as the canonical implementation root field.
- Add `aiqt init --implementation-root` as a preferred alias that writes `existingRepositoryPath`.
- Add `aiqt update --implementation-root` as a preferred alias that writes `existingRepositoryPath`.
- Keep `aiqt update --repository-path` supported for backward compatibility.
- If both `--implementation-root` and `--repository-path` are provided and differ, return invalid input, exit 3.
- Display runtime-derived control root and resolved implementation root in `aiqt status`.
- Include root information in `aiqt manage` output. `aiqt manage` is an established command.
- Inject root information into `aiqt prompt driver`, `aiqt prompt plan`, `aiqt next packets`, and `aiqt prompt checkpoint`.
- Make M15/M15-RC1 source-control guidance consume resolved implementation root automatically.
- Add tests for same-root initialization, explicit split-root initialization, update aliases, status, manage, prompts, packets, and backward compatibility.

7.2 Out of Scope

- Adding `project.paths.implementationRoot` or `project.paths.controlRoot`.
- Persisting `controlRoot` as an absolute path.
- AIQT executing Git commands.
- AIQT automatically creating GitHub repositories, remotes, branches, pull requests, commits, or tags.
- AIQT creating external application directories unless behavior already exists and is explicitly safe.
- Changing checkpoint schema to store verified Git state.
- Packaging, npm publishing, installer generation, or CI release automation.

8. Canonical Root Model

Concept	Definition	Storage / Resolution Rule	Usage
controlRoot	Directory from which AIQT loads .aiqt and where AIQT commands should be run.	Runtime-derived from the loaded .aiqt path. Must not be persisted as an absolute project.json field.	Rendered in prompts/status/manage as the AIQT command root.
implementationRoot	Operational term for where application source code, Git, app commands, validation, and browser checks run.	Resolved from project.existingRepositoryPath if set; otherwise falls back to controlRoot.	Consumed by working-directory, source-control, repository-boundary, validation, and file-reporting guidance.

9. Project Data Model

Do not add a new paths object in M16. The schema model should keep using the existing optional field:

```
{
  "name": "client-portal-for-freelancers",
  "objective": "...",
  "existingRepositoryPath": "../client-portal-for-freelancers"
}
```

- No new required ProjectSchema fields are added.
- No AIQT_SCHEMA_VERSION bump is required because older project.json files remain valid and no new required field is introduced.
- existingRepositoryPath should be documented and rendered as implementation root in user-facing guidance.
- Relative paths are resolved relative to the runtime control root for display/guidance.
- If existingRepositoryPath is missing, resolved implementationRoot = controlRoot.

10. Command Contract Changes

Command	M16 Behavior	Exit Behavior
aiqt init	Initializes AIQT in the current directory. If no implementation root is provided, implementationRoot resolves to controlRoot. existingRepositoryPath may remain unset in same-root projects.	Existing init behavior retained.
aiqt init --implementation-root	Initializes AIQT and writes the supplied path to existingRepositoryPath. This is a clearer alias for configuring implementation root.	Invalid path/input returns invalid input, exit 3, using existing invalid-input semantics.
aiqt update --implementation-root	Updates existingRepositoryPath. Idempotent if the same value is supplied.	Existing update semantics. Invalid input returns exit 3.
aiqt update --repository-path	Remains supported for backward compatibility and writes the same existingRepositoryPath field.	Existing behavior retained. If combined with --implementation-root and values differ, return exit 3.
aiqt status	Displays AIQT control root and resolved implementation root in the project section.	Existing status semantics.
aiqt manage	Displays root summary. Do not claim Git verification unless AIQT actually verifies it in a future milestone.	Existing manage semantics.
aiqt prompt driver / plan / checkpoint	Automatically includes root information and root-specific operating guidance.	Existing prompt semantics.
aiqt next	Packet guidance uses resolved implementationRoot for app/Git/validation/Playwright instructions.	Existing next semantics.

11. Git and Repository Boundary Clarification

- aiqt init does not inspect Git repository boundaries.
- aiqt init does not select or reject parent Git repositories.
- Repository-boundary verification remains agent-executed through M15-RC1 prompt/packet guidance.

- M16 only ensures that the expected implementation root is known and rendered consistently.

12. Root Resolution Rules

- controlRoot = runtime directory containing .aiqt.
- resolvedImplementationRoot = existingRepositoryPath resolved relative to controlRoot when set.
- If existingRepositoryPath is missing, resolvedImplementationRoot = controlRoot.
- All generated guidance should use resolved absolute display paths to remove ambiguity for agents.
- Status/manage JSON output should include stored existingRepositoryPath and resolved implementationRoot if current output conventions support JSON fields.

13. Generated Guidance Requirements

13.1 Driver Prompt

```
AIQT control root:
<resolved-control-root>

Implementation root:
<resolved-implementation-root>

Run AIQT commands from the control root.
Run application, Git, validation, and Playwright/browser commands from the implementation root.
```

13.2 Plan Prompt

- Planning guidance should know whether this is same-root or split-root.
- If source control is required and repository state is unknown, the plan must include or instruct an early repository baseline Work Unit.
- Design-system, component-system, and repository-baseline guidance must coexist deterministically.

13.3 Agent Packet

- Working Directory Discipline should use the resolved roots.
- Source Control Expectations should compare Git top-level against resolved implementationRoot.
- Required Output should ask for changed files relative to implementationRoot.

13.4 Checkpoint Prompt

- Checkpoint guidance should ask for implementation-root-based Git top-level and boundary report.
- File paths in checkpoint output should be relative to implementationRoot unless explicitly reporting AIQT control files.
- Source-control risk report remains self-attested by the agent unless a future milestone adds verification commands.

14. Backward Compatibility

- Existing projects without existingRepositoryPath continue to load and resolve implementationRoot to controlRoot.
- Existing projects with existingRepositoryPath continue to work; M16 changes the user-facing terminology, not the storage field.
- No migration command is required.
- Existing M13/M14/M15/M15-RC1 guidance tests should remain valid, with expectations updated only where root text is intentionally added.

15. Testing Scope

Area	Required Tests
Schema/model	Project config accepts missing existingRepositoryPath and explicit existingRepositoryPath. Invalid types fail existing validation. No paths object is introduced.
Init	aiqt init defaults implementationRoot to controlRoot without needing stored existingRepositoryPath. aiqt init --implementation-root writes existingRepositoryPath.
Update	aiqt update --implementation-root and --repository-path both write existingRepositoryPath. Supplying both with different values returns exit 3.
Status/manage	status and manage display control root and resolved implementation root. JSON mode includes deterministic root fields where applicable.
Prompts	driver, plan, checkpoint prompts include configured roots and no longer require manual root repetition.
Packets	aiqt next packet renders root-specific working-directory and source-control guidance. Changed-file instruction references implementationRoot.
Backward compatibility	old fixtures without existingRepositoryPath work. Old fixtures with existingRepositoryPath are interpreted as implementation root.
Regression	No new public commands. Existing exit codes and runlog event types remain unchanged unless current code already appends events for init/update.

16. Validation Commands

Primary validation:

```
pnpm validate
```

Fallback if blocked by the known ERR_PNPM_IGNORED_BUILDS/esbuild gate:

```
node_modules/.bin/tsc --noEmit
node_modules/.bin/eslint .
node_modules/.bin/vitest run
node_modules/.bin/tsc
```

The implementation report must state which validation path was used and whether pnpm side effects such as pnpm-lock.yaml or pnpm-workspace.yaml were reverted before staging.

17. Acceptance Criteria

- A user can start a normal project by creating a folder, running aiqt init, and relying on AIQT-generated prompts for root discipline.
- aiqt init without --implementation-root resolves implementationRoot to the current control root.
- aiqt init --implementation-root configures split-root projects by writing existingRepositoryPath.
- aiqt update --implementation-root can revise the implementation root by writing existingRepositoryPath.
- aiqt update --repository-path remains supported and compatible.
- status and manage display root information clearly.
- driver, plan, packet, and checkpoint guidance use resolved roots automatically.

- M15/M15-RC1 source-control and repository-boundary guidance consumes implementationRoot instead of relying on manual prompt text.
- No project.paths.implementationRoot or persisted controlRoot is introduced.
- No AIQT Git/GitHub automation is introduced.
- All tests and validation pass.

18. Implementation Report Requirements

- Branch name and commit hash.
- Files changed summary.
- Exact schema/model changes and whether AIQT_SCHEMA_VERSION changed.
- Confirmation that no paths.implementationRoot or persisted controlRoot was introduced.
- Command option changes for init/update/status/manage/prompts/next.
- Root resolution behavior implemented.
- Backward compatibility handling for existingRepositoryPath and missing repository path.
- How prompts and packets consume the root model.
- Validation results and final test count.
- Any deviations from this specification.
- Uncommitted files remaining.

19. Recommended Dogfood Check After M16

After M16 is merged, restart dogfood v4 using only the short startup prompt:

```
Project idea:
Build a full-stack Client Portal for Freelancers.

Use the AIQT CLI for this project.
Create a new folder and run aigt init to start.
Use AIQT for the full workflow from planning to final review.
```

Expected result: the agent should not need a long manual root/context prompt. AIQT should generate and carry the root discipline through planning, packets, checkpoints, source-control reporting, and final review.